

Prediction Market Data Pipeline

Kalshi and Polymarket ingestion, cross-platform analysis, and research synthesis for a weekly internship deliverable

Executive summary

I built a live data pipeline that pulls Kalshi and Polymarket prediction-market data into Supabase on a schedule, and paired it with research into how professional prediction-market traders think about pricing, resolution risk, and cross-platform edges. 19 markets across crypto, macro, and politics get polled on an unattended GitHub Actions cron job, nominally every 2 hours. On the analysis side, I used the collected data to build a repricing-speed metric, compare volume across markets, and measure price divergence across 3 linked market pairs. Along the way I caught a question-polarity mismatch that would have produced a false 80-percentage-point "divergence" on the clearest pair in the dataset, and fixed it before it made it into a finding.

Everything below is a point-in-time analysis of data collected between 2026-09-02 and 2026-09-11 (773 snapshot rows across 19 markets). The cross-platform comparisons in Section 4.3 start from 2026-09-04, since Polymarket ingestion came online after Kalshi's. The pipeline keeps running, so the analysis window is pinned in **analysis/config.py** and every figure quoted here is generated from that window rather than typed in by hand. Re-running the report on a later date reproduces these same numbers until the window is deliberately moved.

1. Architecture and pipeline

1.1 Schema

Six tables live in Supabase (Postgres), with Row Level Security enabled. Writes go through the `service_role` key, which bypasses RLS, so no read policies were needed for this stage of the project.

Table	Purpose
platforms	Kalshi / Polymarket
markets	One row per tracked market, upserted on (platform_id, external_id)
market_snapshots	The core time-series table: one row per market per poll
resolutions	Final outcomes, written by the backfill step once a market settles
cross_platform_links	Manually matched market pairs tracking the same real-world event
kol_theses	Research log of prediction-market researcher theses and frameworks

1.2 Ingestion

Both platforms expose market and price data with no authentication required for reads. Only trading requires auth, and this pipeline never needed it. Kalshi's `/trade-api/v2/markets/{ticker}` and

Polymarket's **/markets/keyset** both return the current price alongside metadata in a single call, so a periodic snapshot poll doesn't need a separate historical-candlestick or order-book call.

The watchlist (12 Kalshi tickers and 7 Polymarket slugs, across crypto, macro, and politics) is a fixed list rather than a dynamically re-selected top-N by volume. A market that drops in and out of tracking from day to day would break the time-series continuity the whole analysis depends on.

One wrinkle worth recording: Kalshi gives every market inside an event the same title, so all five Bitcoin strike buckets arrive as "BTC price on Jan 1, 2027?". The strike sits in a separate **yes_sub_title** field. Until I appended it, five different markets shared one label and the volume chart drew five bars with identical names.

1.3 Scheduling and reliability

A GitHub Actions workflow (**.github/workflows/prediction-markets-w1.yml**, at the repo root, since GitHub Actions doesn't discover workflows nested in a project subfolder) asks for both ingestion scripts every 2 hours. How often that actually happens is its own problem, covered in the next paragraph. The cron is offset from the top of the hour to avoid the queueing delays GitHub applies when many workflows are scheduled for :00 at once. A shared logger (**ingest/logging_config.py**) writes timestamped output to both the console, visible in the Actions logs, and a local **logs/pipeline.log** file, which the workflow also uploads as a build artifact so it survives past the runner. That replaced the four duplicated logging setups the ingestion scripts each carried before.

The requested schedule and the delivered one are not the same thing. Across 42 poll batches over 208 hours, the median gap between batches is 4.6 hours rather than 2, and the worst gap was 34.6 hours. That works out to 4.8 batches a day against the 12 requested, or about 40%. The likeliest cause is GitHub dropping or delaying scheduled events under load, which is documented behaviour for cron on hosted runners rather than anything specific to this repository.

Two caveats keep that number honest. First, 40% is an upper bound on what GitHub delivered, because the batch count includes manual **workflow_dispatch** runs and local runs I made while developing, and snapshot rows don't record which trigger produced them. Separating them needs a run-origin column, which is a fix rather than a caveat, and it opens Section 6. Second, 38 of the 42 batches wrote all 19 markets, not all of them. The 4 exceptions are below. Three fall in the bring-up period before both ingestion scripts ran together, and the fourth is a local Polymarket-only run. None is an unattended failure, but an earlier draft of this report claimed every batch was complete, and that was wrong.

- 2026-09-02 07:58 UTC: 12 markets, kalshi only
- 2026-09-03 18:35 UTC: 12 markets, kalshi only
- 2026-09-03 20:09 UTC: 7 markets, polymarket only
- 2026-09-09 19:37 UTC: 7 markets, polymarket only

After each ingestion run, a resolutions backfill step checks every tracked market against its platform and records the outcome once a market settles. The resolutions table holds 0 rows because nothing tracked has closed yet, but the mechanism is in place and running. A second, independent workflow checks every 6 hours whether the newest snapshot is more than 10 hours old, and fails if it is, which sends GitHub's usual failed-workflow email. The 10-hour threshold sits above the real 4.6-hour median gap, so

a couple of skipped runs don't trip it while a true outage still does.

2. Platform and API differences

- Polymarket's original **/markets** and **/events** endpoints were deprecated and sunset on 2026-05-01, but they still return HTTP 200, with the deprecation visible only in the response headers, so it would have been easy to build on a dead endpoint without noticing. I moved to the replacement, cursor-paginated **/markets/keyset**, before writing any ingestion code.
- Open interest isn't symmetric between the platforms. Kalshi reports it per market. Polymarket only reports it per event, which sums across every sibling market in a multi-market event. Assigning that combined number to a single market would misrepresent it, so Polymarket's `open_interest` stays NULL here rather than holding a misleading figure.
- Polymarket's `outcomePrices` field is the bid/ask midpoint, not the last executed trade price. I checked it against the raw `bestBid` and `bestAsk` on the same response. The midpoint doesn't go stale on a quiet market the way a last-trade price can, so it's the better choice for a periodic snapshot.
- Polymarket slugs aren't stable identifiers the way Kalshi tickers are. Once a market resolves, the same price threshold gets re-listed under a new slug. I found three different "Bitcoin dip to \$60k" slugs for the same nominal question, so the pipeline stores Polymarket's numeric id as `external_id`, and the watchlist slugs still need re-checking every so often.
- Matching strikes don't mean matching contracts. When I tried to link the crypto markets across platforms, the Kalshi ones asked what price BTC or ETH would be on one specific date, while the Polymarket ones asked whether that price was ever touched before a deadline. That's the same asset at a similar dollar figure but a different question, and I only caught it by reading the contract terms instead of trusting the strike price. Section 4.1 shows the difference is measurable, not just semantic.

3. Research synthesis

I logged seven theses from three X accounts, covering theory, trading methodology, and platform resolution risk. These are secondhand accounts recorded in the `kol_theses` table, not claims I verified independently.

3.1 Resolution risk as an unpriced cost

@thenarrator argues that a YES price near \$1 still carries a hidden cost: capital stays locked until settlement, so a near-certain bet can be capital-inefficient if resolution drags or the exit is thin. He thinks markets need a visible "carry" metric, something like expected time-to-redemption shown next to price, similar to a funding rate on perpetual futures. The last few cents near \$1, in his view, often just compensate for resolution or dispute risk the trader hasn't priced in.

@Domahhhh gives four documented cases where that abstract resolution risk played out. Polymarket resolved a Venezuela "invasion" market against its own stated criteria after Trump publicly claimed the US controlled the country. Polymarket resolved an Epstein "blackmail" market YES off one ambiguous email, using a much looser standard than it applied days later. Kalshi paid out a "will Biden meet

Trudeau" market at \$0 even though the meeting was televised, because its rules only recognized two sources and neither reported it. And Kalshi settled an Oscars-viewership market on preliminary Nielsen numbers hours before the final numbers came out and flipped the outcome. Together they make narrator's point less hypothetical.

3.2 Trading methodology: holding against consensus

@Domahhhh's Fed Chair nomination trade shows what acting on that kind of read looks like in size. He held a large net-short position against the market-implied favorite, who reached 85% consensus, for roughly five months and through four different frontrunner swaps, based on his own analysis that the favorite was a poor fit for monetary policy. The eventual pick matched his own, lower-probability estimate instead, netting him \$425,000. It's a contrarian position that paid off because the analysis behind it held up, not just because he refused to fold.

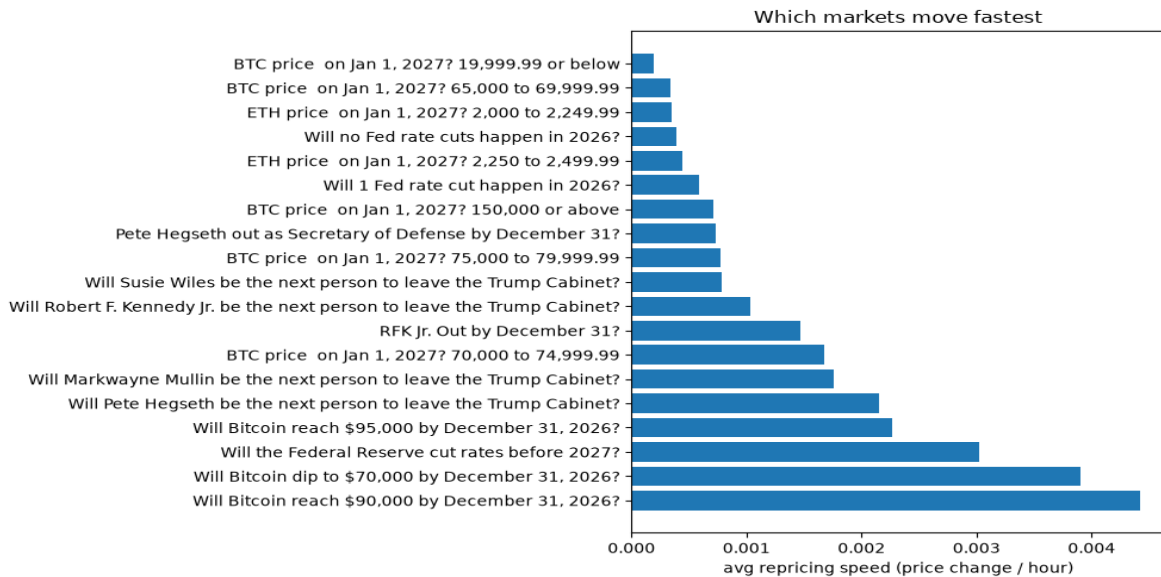
3.3 Theory: why bounded markets discipline claims

@VitalikButerin argues prediction markets are epistemically healthier than social media or unbounded asset markets. A bad take on social media earns unaccountable clout; a bad bet on a prediction market loses real money. And bounded $[0,1]$ pricing resists the reflexivity and pump-and-dump dynamics that unbounded markets are prone to.

4. Findings from the collected data

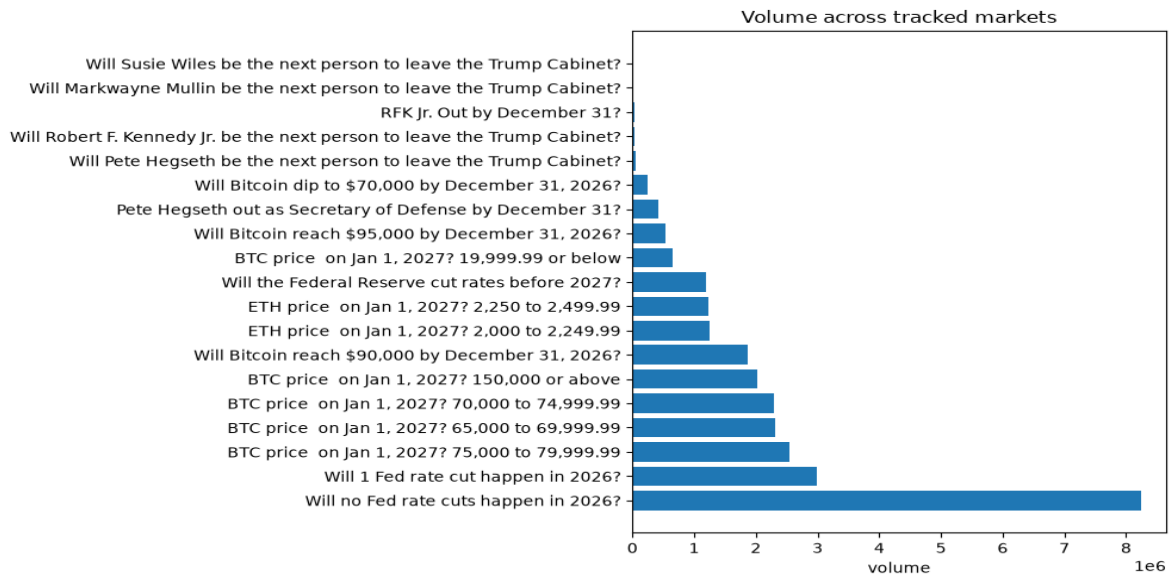
4.1 Repricing speed

Narrator's "prediction-market VIX" framing, made measurable: the mean absolute price change per hour for each market across the window. It's a rough proxy for how much a market is still arguing with itself.



The split is not crypto against politics, which is what I expected going in. It is contract structure. The three fastest markets are "Will Bitcoin reach \$90,000 by December 31, 2026?" at 0.00442 per hour, "Will Bitcoin dip to \$70,000 by December 31, 2026?" at 0.00390, and "Will the Federal Reserve cut rates before 2027?" at 0.00302. The three slowest are all Kalshi price-on-a-date buckets: "BTC price on Jan 1, 2027? 19,999.99 or below" at 0.00020, "BTC price on Jan 1, 2027? 65,000 to 69,999.99" at 0.00033, and "ETH price on Jan 1, 2027? 2,000 to 2,249.99" at 0.00035. Bitcoin sits at both ends of that ranking. A touch-by-deadline contract reprices on every move toward the barrier, while a settle-on-one-date bucket only cares where price finishes, so it ignores the same volatility. That is the Section 2 contract-type distinction showing up as a number.

4.2 Volume distribution

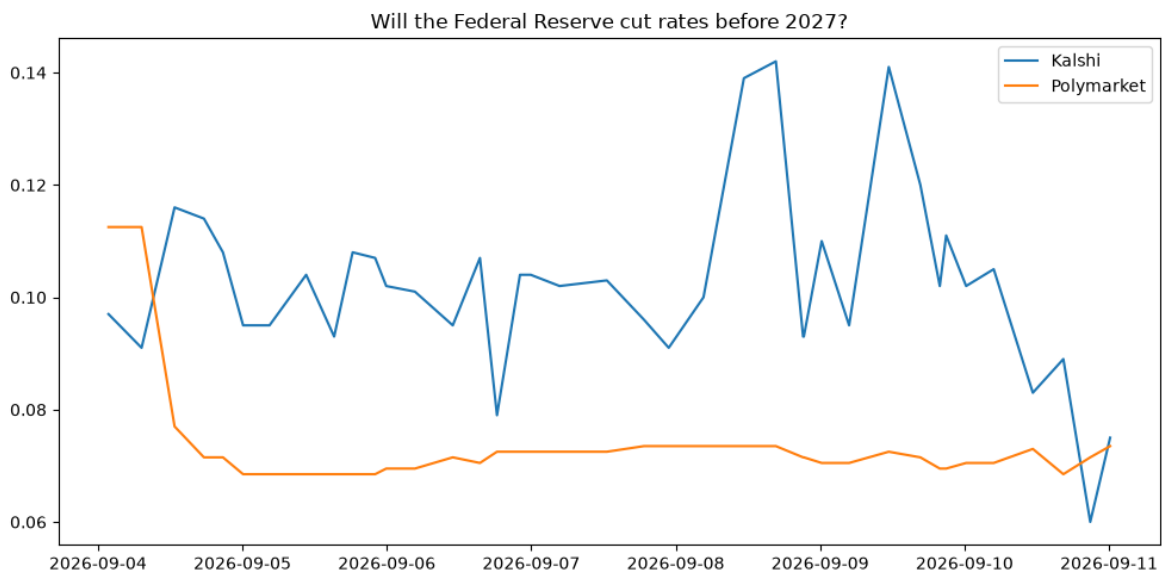


The Polymarket "Will no Fed rate cuts happen in 2026?" market carries about \$8.24M in volume, against \$18,148 and \$14,114 on the thinnest Kalshi cabinet markets. That market also sits 16th of 19 on repricing speed, so the heaviest book in the set is among the slowest to move, though it is not the single slowest. Evercore ISI's analysis of five years of completed Kalshi and Polymarket markets reportedly found that high-volume markets price more reliably than thin ones. I could not verify that study directly, so treat it as context from the research log rather than as evidence this dataset establishes.

4.3 Cross-platform divergence

Of five intended cross-platform pairs, only three are populated in `cross_platform_links`: one macro pair and two politics pairs. I didn't add crypto pairs. Checking the actual contract terms, not just the dollar strikes, showed that the tracked Kalshi crypto markets ask what price BTC or ETH will be on a specific future date, while the tracked Polymarket markets ask whether that price is ever touched before a deadline. Those are different questions even at matching strikes, so linking them would have produced a divergence number that looks real but doesn't measure anything.

4.3.1 Macro pair: Fed rate cut



This compares Kalshi's "Will the Federal Reserve cut rates before 2027?" against Polymarket's "Will no Fed rate cuts happen in 2026?", with Polymarket's series inverted for comparison. That inversion mattered: the two questions are logically opposite (on Kalshi, Yes means a cut happens; on Polymarket, Yes means it doesn't), so a naive raw comparison showed a false 80-percentage-point "divergence" that was really just two inverted questions lined up against each other. Once corrected for polarity, the series open on 2026-09-04 at 0.097 on Kalshi and 0.113 on Polymarket, separate through the middle of the window, then close on 2026-09-11 at 0.075 and 0.074. Both drifted down and finished within a thousandth of each other, after Kalshi spiked to 0.142 mid-window and gave it back. Across 39 aligned snapshots the Kalshi:Polymarket ratio averaged 1.40 (median 1.43, standard deviation 0.26, range 0.81 to 1.94). The divergence is real inside the window but does not survive to the end of it, which is the strongest argument in this report against reading a durable pattern into one week of data.

4.3.2 Politics pairs: RFK Jr. and Hegseth



The two politics pairs don't show one uniform pattern, and treating them as if they did would overstate the finding. The RFK Jr. pair (Kalshi:Polymarket ratio) is tight and consistent: mean 0.70, median 0.67, standard deviation 0.06 across 39 points, ranging only from 0.64 to 0.78. Kalshi prices this market consistently below Polymarket. The Pete Hegseth pair is higher and noisier: mean 0.85, median 0.88,

standard deviation 0.12, ranging from 0.59 to 1.07. That upper bound is above 1.0, meaning Kalshi at times priced Hegseth's departure higher than Polymarket did. A single structural story about Kalshi's comparative "next to leave" framing sitting below Polymarket's independent yes/no framing fits RFK Jr. well and fits Hegseth badly.

5. Limitations

- 19 markets tracked, a deliberately small and fixed watchlist, not the full catalog of either platform.
- Only 3 of 5 planned cross-platform links are populated. The 2 crypto pairs were never added: the tracked Kalshi and Polymarket crypto markets turned out to be different contract types (price-on-a-date versus touch-by-deadline), not just different strikes.
- 8.7 days of observation (2026-09-02 to 2026-09-11), and less than that for the cross-platform pairs, which start 2026-09-04. The Fed-pair divergence is a real, measured result, but the series converge again by the end of the window, so treating the mid-window gap as a durable platform difference would be reading too much into one week.
- The politics pairs are approximate matches, and the mismatch is wider than the tickers suggest. Kalshi's cabinet-departure tickers are named KXCABOUT-26MAY22, but their actual close_time is 2029-01-20, the end of the presidential term, while the matched Polymarket markets close 2026-12-31. Same underlying question, very different windows. Reading the date out of the ticker string instead of the close_time field is what put a wrong figure in an earlier draft of this report.
- Polymarket's open_interest isn't available at market granularity (see Section 2), so it's NULL throughout.
- Snapshot rows don't record what triggered the run, so scheduled, manual and local runs are indistinguishable in the delivery statistics in Section 1.3.

6. Next steps

- Record a trigger source on every snapshot row (scheduled, manual, or local) so the GitHub delivery rate in Section 1.3 can be measured against scheduled runs alone instead of against every write to the table.
- Track a crypto pair the two platforms define the same way. That means finding a Kalshi market phrased as touch-by-deadline, since the ones tracked now ask a different question than the Polymarket side. Section 4.1 suggests the two contract types behave differently enough that pairing them would measure the contract, not the platform.
- Trade-level ingestion and wallet-level "smart money" tracking through Polymarket's subgraph. This was scoped out of this week's MVP and flagged as a stretch goal from the start.
- Widen the watchlist and extend the window once the pipeline has run unattended for longer, to test whether the Fed-pair convergence holds or was a one-week artifact.